
MACHINE LEARNING OPERATIONS
NOVA IMS

Automated Property Price Prediction

Production-ready MLOps Pipeline

Bárbara Franco	20250388
Inês Caetano	20221950
Maria Carvalho	20221953
Maria Miguel Fonseca	20250380
João Bernardo	20221889

Contents

1	Abstract	1
2	Introduction & Problem Statement	1
2.1	Success Metrics	1
3	Project Planning	2
4	Pipeline Architecture	2
5	Data Exploration & Feature Engineering	3
5.1	Data Unit Tests	3
5.2	Preprocessing Pipeline	4
6	Modeling Results	4
6.1	Explainability	5
7	Production Considerations	5
7.1	Deployment & Orchestration	5
7.2	Experiment Tracking & Model Governance	5
7.3	Reproducibility & Reliability	6
7.4	Data-drift monitoring	6
7.5	Risks & Mitigations	6
8	Packages & Versions	8
A	Appendix	9

1 Abstract

This project presents an end-to-end Machine Learning Operations pipeline for predicting residential property prices in Portugal from the Portugal listings dataset, framed as a regression problem on the listing Price. Rather than focusing solely on predictive accuracy, our goal was to operationalise the full model lifecycle following MLOps best practices. The solution is built with Kedro as the orchestration framework and integrates a modular set of tools: Great Expectations for multi-stage automated data validation with a traffic-light quality gate, anti-leakage feature engineering with scalers and encoders fitted on training data only, RFE for feature selection, Optuna for hyperparameter tuning, and the MLflow Model Registry for experiment tracking and champion model management. Explainability is provided through SHAP, data drift is monitored with Evidently, features are managed in a Hopsworks feature store, and the workflow is scheduled with Prefect and containerised with Docker for reproducibility. The champion model (Random Forest) achieved a validation R^2 of 0.72 and an out-of-sample test R^2 of 0.68, demonstrating a reproducible, monitored, and maintainable machine learning system.

2 Introduction & Problem Statement

The Portuguese real estate market has experienced significant price volatility in recent years, making accurate property valuation both practically relevant and technically challenging. This project addresses the problem of automated property price prediction using a dataset of approximately 135,000 listings scraped from a major Portuguese real estate portal.

The dataset contains features spanning location (district, city, town), property type, energy certificate rating, area measurements, room counts, and construction year. Its scale and diversity introduce realistic MLOps challenges: high-cardinality categorical variables, substantial missing data, and a heavily right-skewed price distribution. The target variable is property price in EUR.

The goal is to build a production-ready ML pipeline, not merely a model, covering data ingestion, validation, preprocessing, feature selection, model training, explainability, and drift detection, following MLOps best practices.

2.1 Success Metrics

A model that performs well but cannot be reliably deployed, monitored, or reproduced has limited real-world value. For this reason, we define success on two levels.

Pipeline-level:

- the full Kedro pipeline runs end-to-end without manual intervention;
- data unit tests (Great Expectations) validate data quality at key pipeline checkpoints;
- all experiments are tracked and models versioned in MLflow;
- drift detection produces actionable reports;
- code unit tests (pytest) cover the core functions of each pipeline stage, targeting 100% coverage on main nodes.

Model-level:

Metric	Scale	Purpose
RMSE	$\log(\text{Price})$	Primary tuning objective
MAE	$\log(\text{Price})$	Interpretable absolute error
R^2	—	Skill above mean baseline

The model is considered successful if it meaningfully outperforms a mean predictor baseline (DummyRegressor) across all three metrics on the held-out validation set.

3 Project Planning

We organised the project using a scrum-inspired approach, dividing the work into five sprints with clear goals (Table 5). This helped us manage time effectively and keep all pipeline components integrated. Tasks were assigned at the start of each sprint, with regular check-ins to review progress and address blockers quickly. We used Git for version control and collaboration, enabling efficient tracking of changes and seamless teamwork throughout the development process.

4 Pipeline Architecture

This section outlines the modular structure of our machine learning pipeline, implemented in Kedro and visualised with Kedro-Viz (Figure 1). The system is organised into five composable pipelines:

- **Data preparation** (`data_prep`) runs ingestion, carves a reproducible out-of-sample split (`learning_data` / `test_data`), cleans the data, performs the train/validation split, and applies feature engineering (imputation, capping, target encoding and scaling, all fitted on the training set only). It ends with a Great Expectations validation step that writes a traffic-light flag, gating the downstream stages on data quality.
- **Training** compares candidate models, tunes the champion with Optuna, and registers it in the MLflow Model Registry.
- **Inference** applies the train-fitted (champion) to the out-of-sample batch and computes the honest test metrics with the champion.
- **Monitoring** measures data drift with Evidently, and **Reporting** consolidates metrics, data-quality results, drift and SHAP explanations into a single report.

This modular design enforces a clear separation of concerns and reproducibility: each pipeline can be run in isolation or chained into the default end-to-end run.

Table 1: Kedro pipelines and their roles

Pipeline	Role
Ingestion	Loads raw CSV or retrieves from Hopsworks feature store
Data Unit Tests	Great Expectations validation at three tiers
Split Data	Separates learning data from out-of-sample batch
Preprocessing Train	Leakage-safe cleaning applied before the train/val split
Split Train	Divides learning data into train and validation sets
Preproc After Split	Post-split transforms fitted on train only (imputation, capping, encoding, scaling)
Model Selection	Trains 4 candidate models, selects champion by validation RMSE, fine-tunes with Optuna, logs all to MLflow
Feature Selection	RFE on champion model to identify most predictive features
Model Train	Retrains champion on selected features and registers in MLflow
Model Predict	Runs inference on out-of-sample batch using production model
Explainability	SHAP analysis on champion model
Data Drift	Evidently reports comparing training distribution to batch
Reporting	Aggregates GX validation results and drift reports

5 Data Exploration & Feature Engineering

The dataset contains 135,536 property listings across 25 features. The dominant property types are Apartment (53%) and House (31%), with the remainder spread across commercial, land, and other types. The data covers all Portuguese districts, with Lisboa and Porto being the most represented.

Missing data is extensive: 36.5% of all cells are missing, requiring a feature-by-feature strategy. The most incomplete columns are *BuiltArea* (80.4%), *GrossArea* (79.6%), *PublishDate* (78.5%), and *LotSize* (70.8%). *PublishDate* was identified as a scraping artefact with no reliable temporal signal and was dropped entirely. *ConservationStatus* (85.8%) and *Floor* (79.4%) were also dropped. *ConstructionYear* (35.1% missing) and room counts were retained and imputed. A total of 6.6% of rows were exact duplicates and were removed, leaving 126,525 rows. Impossible values (negative areas, prices ≤ 0) and implausible values (>20 bedrooms, >50 rooms) were flagged and set to NaN. Listings located outside Portugal were also removed, keeping the project’s scope focused on the Portuguese market.

The price distribution is heavily right-skewed, with a mean around €370,000 and extreme outliers reaching €1.38 billion. A log1p transformation was applied to stabilise variance and normalise the distribution for modelling.

Spearman correlations show *NumberOfBathrooms* (0.60) and room counts as the strongest price predictors, with area features correlated with each other. Among categorical features, Cramér’s V analysis revealed that Type is the primary driver of association, most notably with *Elevator* ($V=0.70$) and *HasParking* ($V=0.53$), reinforcing its importance as a key modelling feature.

5.1 Data Unit Tests

Data quality validation is enforced at three checkpoints using Great Expectations:

- **Raw data:** structural checks on ingestion (price > 0 , plausible construction year ranges,

valid category sets).

- **Cleaned data** (post-preprocessing, pre-split): correct column types, valid numeric ranges and categories, dropped columns removed, and no nulls in critical columns.
- **Model input** (X_{train}/X_{val} , post-split): no missing values, correct types, ranges consistent after imputation and capping, and row count consistent with the split.

Each checkpoint produces a validation report and writes a traffic light flag (OK/FAIL), ensuring data quality issues are caught early and do not silently propagate through the pipeline.

5.2 Preprocessing Pipeline

Preprocessing is implemented as a Kedro pipeline split into two stages. **Pre-split steps** (applied to the full dataset before splitting) include duplicate removal, column dropping, impossible and implausible value correction, energy certificate consolidation, boolean encoding, ordinal encoding of energy ratings, and log1p transformation of area features. **Post-split steps**, fitted exclusively on the training set, include group-based median imputation by property *Type* with global fallback for unseen types, percentile capping of outliers, target encoding of *District* and *Type*, and standard scaling of all numeric features.

The batch preprocessing path applies the same cleaning and transformation logic as training, with all transforms fitted exclusively on the training set and applied via transform only to new data.

6 Modeling Results

Four candidate models were trained and ranked by validation RMSE on the log-price target (the selection metric, the lower, the better). All candidates share the same feature-engineered training set.

Table 2: Model selection results

Model	R^2	MAE (log)	RMSE (log)
Random Forest	0.711	0.419	0.6645
XGBoost	0.705	0.455	0.6713
LightGBM	0.690	0.473	0.6878
Gradient Boosting	0.513	0.644	0.7373

Random Forest was selected as the best candidate. Before hyperparameter tuning, it was subjected to Recursive Feature Elimination (RFE), which reduced the feature space from 18 to the 15 most informative features. The model was then tuned with Optuna (30 trials) on these selected features, improving validation RMSE from 0.6645 to 0.6513. Consequently, this model was registered in the MLflow Model Registry as the champion model. Finally, we apply it to predict on test data (Appendix, Table 6). The model beats the DummyRegressor(mean) baseline ($R^2 = 0$, RMSE(log) = 1.235). The RMSE (in euros) is inflated by a heavy right tail of luxury-property prices, so R^2 and MAE are reported as the headline metrics.

6.1 Explainability

To interpret the champion model we computed SHAP values over a 200-row sample of the validation set. Figure 2 shows, for each feature, the magnitude and direction of its contribution to the predicted log-price; Figure 3 ranks features by mean absolute SHAP value. The dominant price drivers are:

- **Number of Bathrooms** (the strongest driver): more bathrooms push the prediction towards higher prices, a reliable proxy for property size and standing.
- **District** (target-encoded location) and **Type**: high-value districts and property types shift predictions upward.
- **EnergyCertificate** and **area** features (Total area, Living Area, Lot size): better efficiency ratings and larger areas increase the predicted price.

7 Production Considerations

This project is a proof of concept, but it was built with production deployment in mind. The present section discusses how the pipeline would operate in a real environment, the advantages of the chosen technologies, and the main risks with their mitigations.

7.1 Deployment & Orchestration

The pipeline is fully modular: each Kedro component can run independently or as part of a larger sequence, which mirrors how MLOps systems separate concerns in production. This modularity also makes errors far easier to trace and isolate, a failure can be pinpointed to a specific pipeline rather than the whole system, which simplifies debugging, correction, and collaboration across team members working on different components. **Pipeline behaviour** is fully driven by versioned configuration files (conf/base, conf/test), where preprocessing rules, validation thresholds, model candidates, and hyperparameter search spaces are defined declaratively. This allows the pipeline to be adapted across environments such as development, testing and production by swapping configuration rather than modifying code. **Orchestration** is handled by Prefect, where flows are defined with explicit dependencies, for example ensuring training only runs after preprocessing completes. The result is a containerised pipeline (Docker, addressed in 6.3), ready for deployment to a container platform.

7.2 Experiment Tracking & Model Governance

MLflow provides experiment tracking and model versioning. Every candidate model is logged with parameters and metrics, and the Model Registry implements automatic champion/challenger promotion: a newly trained model only replaces the production champion if it outperforms it on validation RMSE. This gives a clear, auditable model governance process which is essential in production where models are retrained regularly.

7.3 Reproducibility & Reliability

Reproducibility is enforced on two distinct fronts:

- **Environment reproducibility:** `uv` provides deterministic dependency management, so every teammate and the production environment install identical package versions. This directly supports the requirement that anyone can run the pipeline and obtain the same results.
- **Containerization:** `Docker` packages the full environment: OS, system libraries, and dependencies. This guarantees the pipeline runs identically across machines and is ready for deployment to any container platform.

Beyond reproducibility, **reliability** is supported by unit tests (`pytest`) covering the pipeline’s core functions, and by the three-tier Great Expectations data validation described in Section 3. Together these ensure issues are caught before they reach production.

7.4 Data-drift monitoring

Once a model is in production, its inputs can drift away from the data it was trained on, silently degrading accuracy. We monitor this with Evidently, comparing two distributions: a reference, from our learning data (the training pool the model learned from); and the current batch, test data (the incoming out-of-sample batch). We monitor a curated set of 15 features (12 numeric + 3 categorical), deliberately excluding fields with very high cardinality like City/Town.

Drift is measured per feature with a distribution-distance test and flagged against a threshold of 0.1. A dataset-level alarm is raised only if the share of drifted features is equal to or higher than 0.5. The pipeline outputs a drift table per feature and an interactive HTML report, and feeds the verdict into the consolidated report.

Comparing the real test data against learning data, 0 of 15 features drifted. All distances were well below the threshold. This is the expected result as the train/test split is random. As a stress test, the pipeline can inject artificial drift by multiplying the numeric features of the batch by $\times 1.3$, to verify that the detector correctly identifies distributional shifts. With injection enabled, the detector correctly flagged 5 of 15 features as drifted (Table 7). Because only 33% of features crossed the threshold (below the 50% dataset-level threshold), the dataset-level verdict remained no drift. A lower share threshold or a larger shift would escalate it to a global alarm.

7.5 Risks & Mitigations

Table 3: Identified risks and proposed mitigations

Risk	Mitigation
Scalability. The pipeline currently uses <code>pandas</code> (single-machine). With $\sim 135k$ rows this is efficient, but performance would degrade as data scales to millions of listings.	Migrate processing-heavy nodes to a distributed framework such as Spark or Polars.

Monolithic container. The pipeline currently runs in a single Docker container bundling training and serving dependencies. Sufficient for a proof of concept but limits independent scaling and produces a heavier serving image than necessary.

Model staleness. Property prices drift over time (market shifts, seasonality, macroeconomic factors), so a model trained on historical listings gradually loses predictive accuracy.

Heterogeneous property types. A single global model is used to predict prices across very different property types (e.g. a parking space vs. a luxury apartment) that differ in price dynamics and feature relationships, so one model produces poor predictions for atypical or under-represented types.

Split into separate containers, a lightweight inference service and a heavier training image, orchestrated (e.g. with Kubernetes) to allow each to scale independently.

Continuously monitor data drift with Evidently (reference `learning_data` vs. incoming batches); a detected drift triggers scheduled retraining on fresh data, with champion/challenger promotion managed in the MLflow Model Registry so the deployed model stays current.

Apply a divide-and-conquer strategy: segment listings by property type and train a specialised model per segment, routing each prediction to the corresponding model. This reduces within-model heterogeneity and improves accuracy on minority segments.

8 Packages & Versions

Table 4: Project dependencies and their purpose

Package	Version	Purpose
kedro	1.3.1	Pipeline framework / project structure
kedro-datasets	9.4.0	Catalog datasets (CSV, Pickle, JSON, Matplotlib)
kedro-mlflow	2.0.2	Kedro ↔ MLflow integration (tracking + registry)
kedro-viz	12.4.0	Pipeline visualization (Figure 3)
kedro-telemetry	0.7.0	Kedro usage telemetry
mlflow	3.11.1	Experiment tracking + Model Registry
optuna	4.8.0	Hyperparameter tuning (TPE)
scikit-learn	1.8.0	Regression models + metrics
shap	0.52.0	Explainability (feature importance)
great-expectations	1.16.1	Data validation / quality (traffic light)
evidently	0.7.21	Data drift monitoring
hopsworks	5.0.0	Feature store
prefect	3.6.27	Workflow orchestration (scheduling)
pandas	2.3.3	Data manipulation
numpy	2.3.5	Numerical computing
matplotlib	3.10.0	Plotting (SHAP / EDA figures)
ydata-profiling	4.18.1	Automated EDA profiling
xgboost	3.3.0	Candidate model
lightgbm	4.6.0	Candidate model
confluent-kafka	2.14.0	Streaming ingestion (Hopsworks)
docker	7.1.0	Containerized reproducible runs

A Appendix

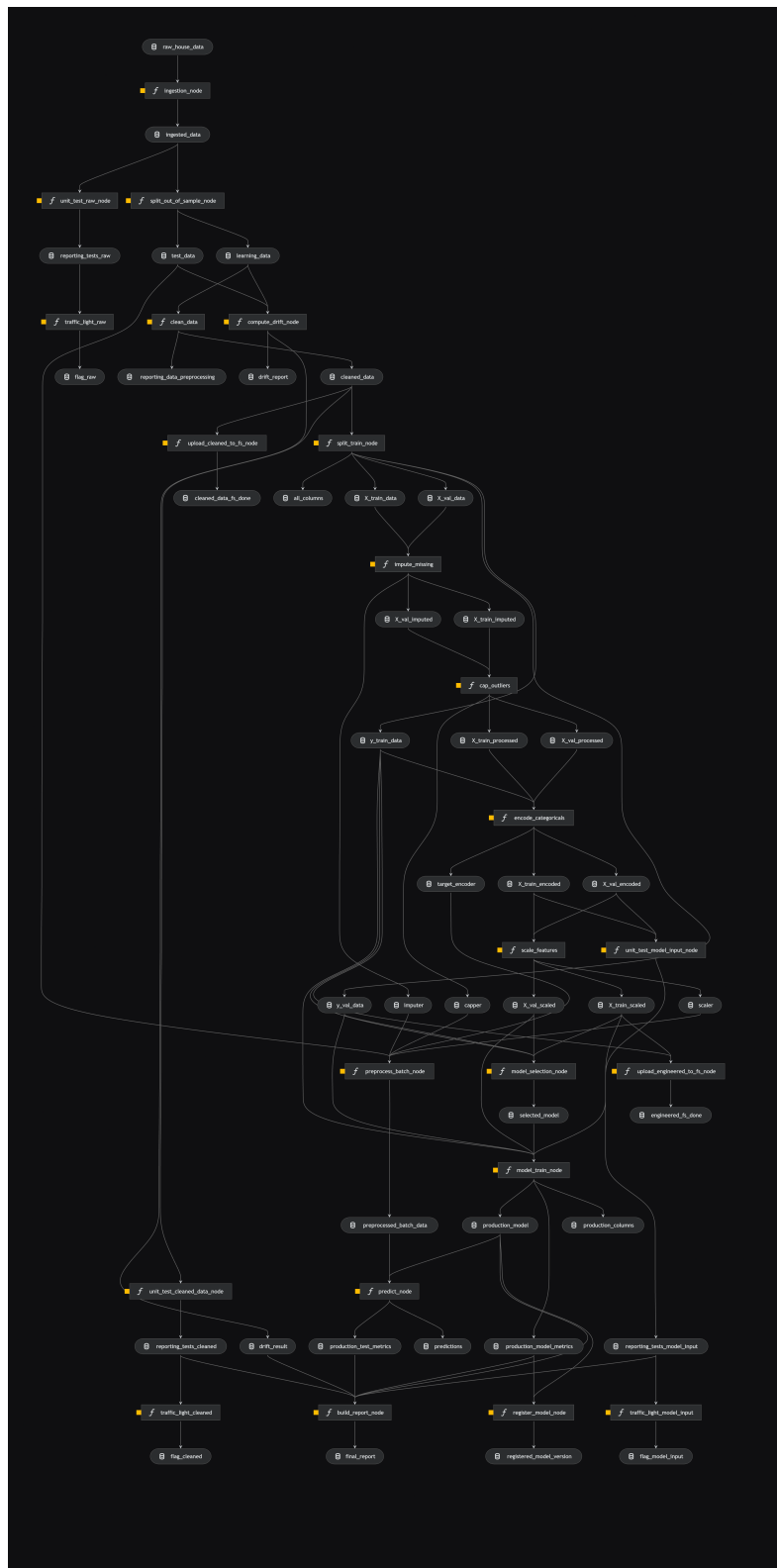


Figure 1: End-to-end Kedro pipeline visualised with Kedro-Viz.

Table 5: Project sprints, focus areas and technologies

Sprint	Focus	Technologies
Sprint 1: Data Foundation	Data ingestion, initial EDA, data quality validation, feature store setup	Kedro, Great Expectations, Hopsworks
Sprint 2: Preprocessing & Feature Engineering	Data preprocessing, feature engineering through encoding and scaling	Kedro
Sprint 3: Model Development	Model selection across candidates, hyperparameter tuning with Optuna	Kedro, MLflow, Optuna
Sprint 4: Evaluation & Explainability	Feature selection (RFE on champion), model retraining on selected features, SHAP explainability	Kedro, MLflow, SHAP
Sprint 5: Monitoring & Delivery	Data drift detection, pipeline orchestration, reporting, containerisation	Kedro, Evidently, Prefect, Docker

Table 6: Random Forest validation and test metrics

Metric	Validation	Test
R^2 (log)	0.722	0.678
RMSE (log)	0.651	0.698
MAE (log)	0.417	0.453
RMSE (€)	-	440 031
MAE (€)	-	138 394

Table 7: Artificial drift detection

Feature	Distance (normed)	Drift
ConstructionYear	27.37	Yes
NumberOfBedrooms	0.451	Yes
NumberOfBathrooms	0.266	Yes
Parking	0.202	Yes
NumberOfWC	0.137	Yes
Price	0.028	No
GrossArea / TotalArea	< 0.1	No

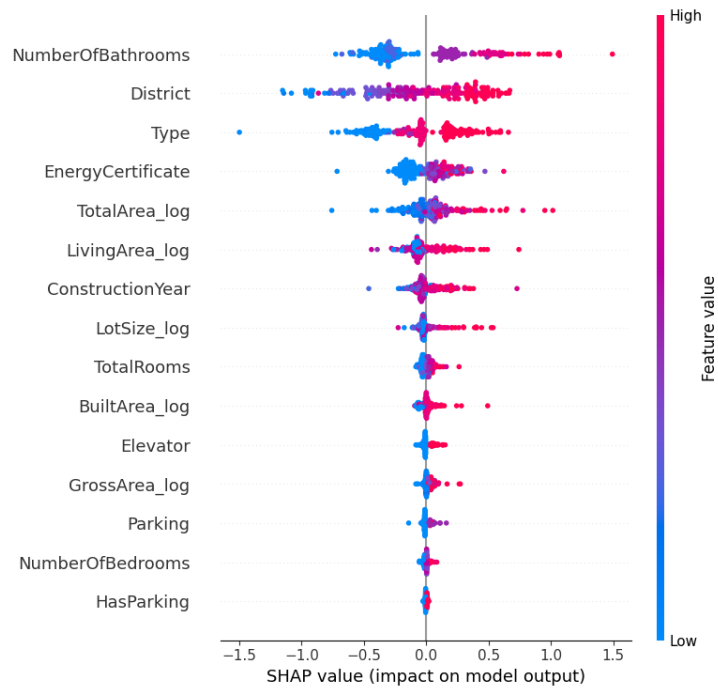


Figure 2: Magnitude and direction of feature's contribution to the model

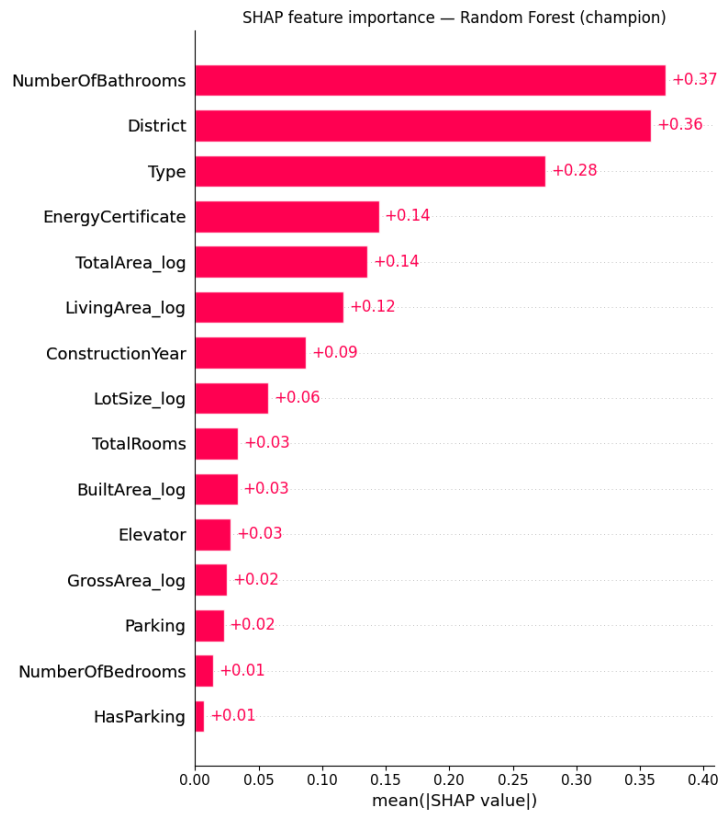


Figure 3: SHAP feature importance: features ranked by absolute SHAP value